

PARALLEL COMPUTING

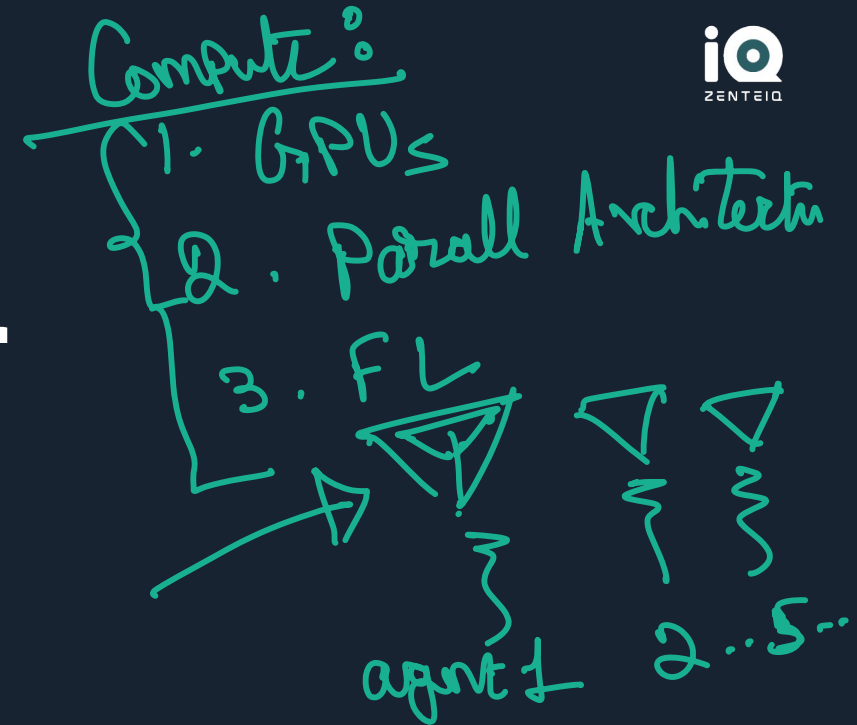
Parallel Computer Architectures

SMP · DMP · Hybrid · OpenMP · MPI · GPU Accelerators

SMP — Shared Memory

DMP — Distributed Memory

Hybrid Architectures



Classification of Parallel Computers

Three fundamental models — differing in memory access and programming model

Parallel Computer
World's Fastest Supercomputer (Frontier, Nov 2025):

1. Param A

2. Watson

$$\begin{aligned} \# \text{Cores} / \text{CPUs} &= \underline{\underline{75000}} \quad ? \\ &= 1\text{-M?} \end{aligned}$$

Classification of Parallel Computers

Three fundamental models — differing in memory access and programming model

World's Fastest Supercomputer (Frontier, Nov 2025):

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX235a, AMD 4th Gen EPYC 240 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/USDA/NSF LINCC United States	11,340,000	1,809.00	2,821.10	9,685
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 4th 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/USDA/NSF Lawrence National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/USDA/NSF Lawrence National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN Fraunhofer IPT, Germany	4,801,344	1,000.00	1,226.28	15,794
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

TOP 500

Classification of Parallel Computers



Three fundamental models — differing in memory access and programming model

2020 — 2020

World's Fastest Supercomputer (Frontier, Nov 2025):

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 26Hz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER BOOSTER - BullSequana XH3000, GH Superchip 72C 18GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

Countries	Count	System Share (%)	Rmax (GFlops)	Rpeak (GFlops)	Cores
1 United States	171	34.2	6,961,330,490	11,002,146,951	58,653,128
2 Japan	43	8.6	1,425,992,450	1,856,582,938	13,615,160
3 China	40	8	204,933,614	343,819,382	13,617,624
4 Germany	40	8	1,396,125,580	1,821,152,384	8,831,292
5 France	23	4.6	334,639,840	514,248,244	4,493,824
6 Canada	19	3.8	158,216,260	242,003,075	1,990,192
7 Italy	18	3.6	888,816,920	1,149,785,108	7,716,656
8 South Korea	15	3	323,114,900	442,276,740	2,799,412
9 Brazil	10	2	124,511,650	242,547,142	1,158,880
10 Taiwan	10	2	208,612,520	300,088,670	1,063,168
11 Norway	9	1.8	44,075,210	62,065,550	642,528
12 United Kingdom	9	1.8	378,120,110	522,317,268	2,548,776
13 Poland	8	1.6	70,980,670	110,451,520	440,000
14 Sweden	8	1.6	66,959,520	95,140,566	451,200
15 Netherlands	7	1.4	291,308,450	489,421,170	1,403,296
16 Saudi Arabia	7	1.4	216,513,210	305,060,750	3,009,856
17 India	6	1.2	113,495,990	141,752,655	996,144
18 Russia	5	1	68,979,000	93,778,830	657,104

Prof. Sashikumar Ganesan | IISc Bangalore

Param Computers

Classification of Parallel Computers

Three fundamental models — differing in memory access and programming model

①

Shared Memory Parallelism (SMP)

- All CPUs share the same memory pool
- Uniform Memory Access (UMA)
- Programmed with OpenMP
- Work decomposition — not data
- Synchronisation can be implicit

②

Distributed Memory Parallelism (DMP)

- Each CPU has its own private memory
- Non-Uniform Memory Access (NUMA)
- Communicate via MPI over network
- User controls data AND work distribution
- Explicit send/receive between processes

③
① + ②

Hybrid Architectures

- SMP within each compute node
- DMP across nodes (node interconnect)
- GPU/Xeon-Phi accelerators per node
- MPI + X (X = OpenMP / CUDA / HIP)
- Most modern supercomputers use this

§1

Shared Memory Parallelism

SMP · Uniform Memory Access · OpenMP

Symmetric Multi Processing

Shared Memory Parallelism (SMP)

All CPUs connect to all memory banks at equal speed

Key Characteristics

Uniform Memory Access:

Every CPU connects to every memory bank with the same access speed

Single Address Space:

All processors see the same global memory — no data distribution needed

OpenMP Implementation:

Uses compiler directives (`#pragma omp`) to define parallel work regions

Work Decomposition:

Programmer specifies HOW work is split — NOT how data is distributed

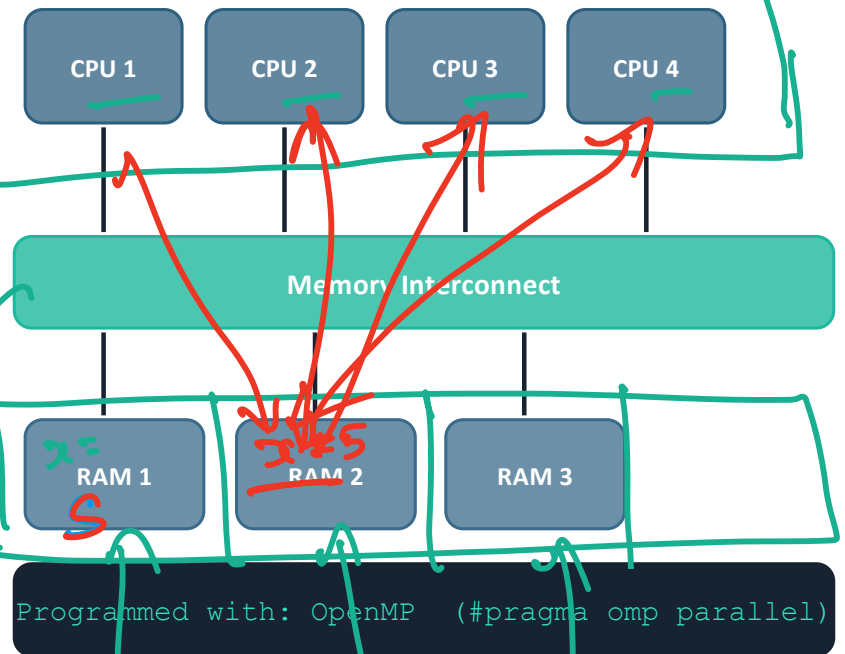
Implicit Synchronisation:

Threads synchronise automatically at parallel region boundaries (can also be user-defined)

Scalability Limit:

Memory bus becomes bottleneck beyond ~64 CPUs — where DMP takes over

SMP Architecture



Shared

CPU 1
32

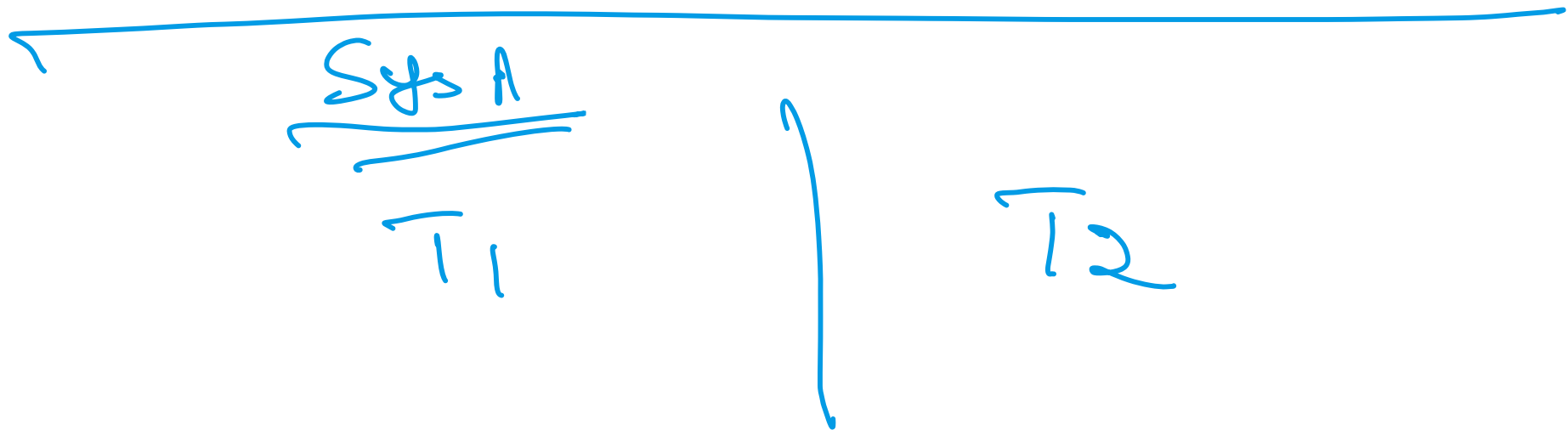
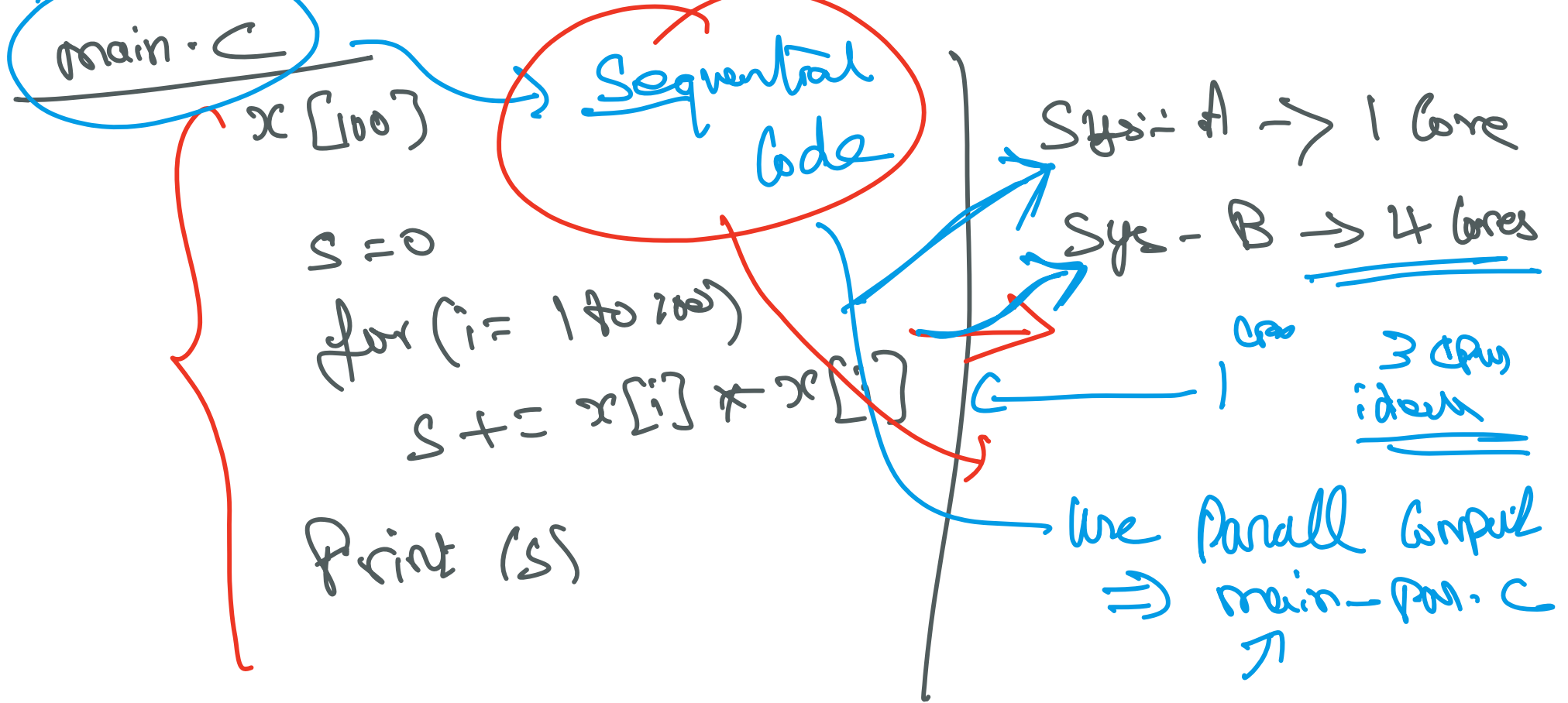
8GB

8GB

8GB

24 GB RAM

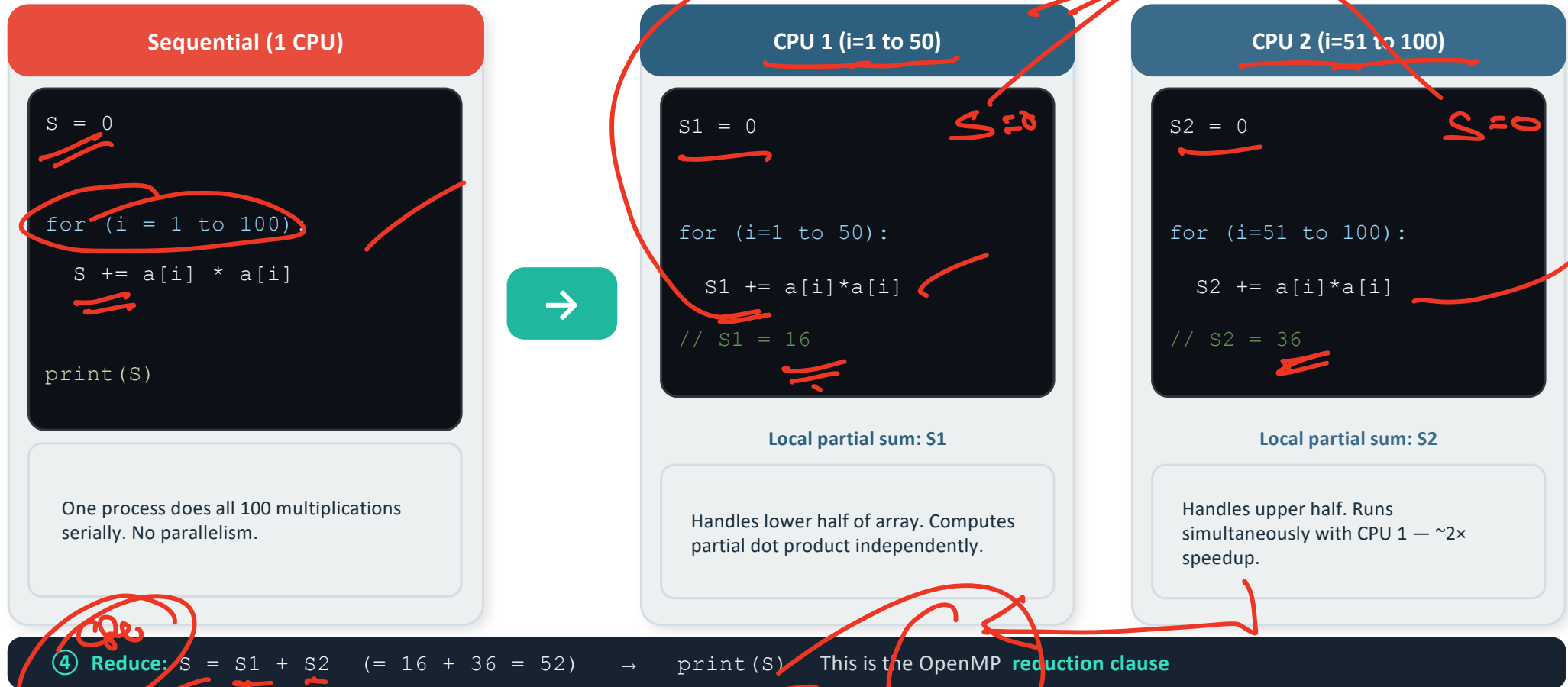
omp sem/lock
22 Core CPUs





SMP in Action — Dot Product Example

Sequential → Parallel with 2 CPUs using work decomposition



OpenMP — Shared Memory Directives

Compiler directives that parallelise C/C++/Fortran without rewriting code

```
#include <omp.h>
```

```
// Parallel region — team of threads
```

```
#pragma omp parallel
```

```
{
```

```
    // Each thread runs this block
```

```
}
```

```
// Parallel for loop (work decomposition)
```

```
#pragma omp parallel for reduction(+:S)
```

```
for (int i=0; i<100; i++) {
```

```
    S += a[i] * a[i];
```

```
}
```

```
// Control thread count
```

```
omp_set_num_threads(4);
```

```
// or: OMP_NUM_THREADS=4 ./myprogram
```

#pragma omp parallel

Creates a team of threads. All threads execute the block.

#pragma omp for

Distributes loop iterations across the thread team (work decomp).

reduction(+:S)

Each thread has private S; results are combined at the end.

private(x)

Variable x is private to each thread — no sharing.

shared(a)

Variable a is shared — all threads read/write same memory.

barrier

All threads wait here until every thread reaches this point.

Rule: Specify WORK decomposition. NOT data decomposition. Data is globally shared.

§2

Distributed Memory Parallelism

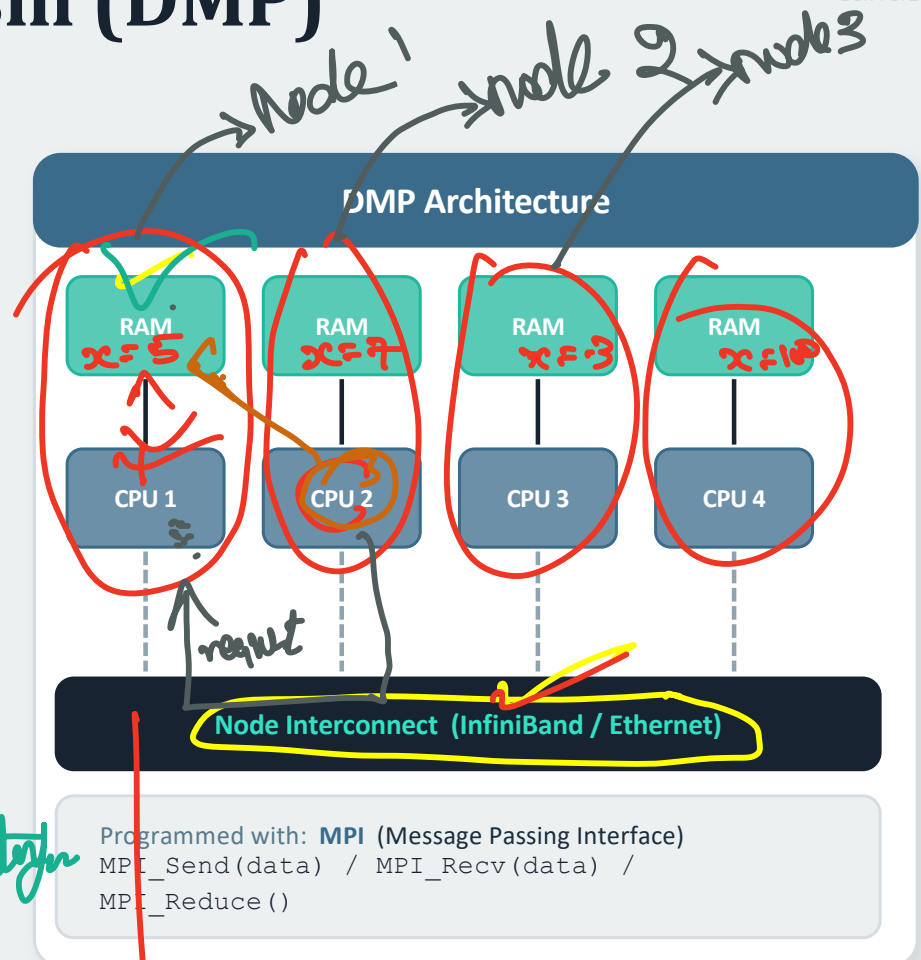
DMP · Non-Uniform Memory Access · MPI

Distributed Memory Parallelism (DMP)

Each CPU has private memory — communicate explicitly via MPI

SMP vs DMP — Side by Side

Property	SMP	DMP
Memory (RAM)	Shared (all CPUs)	Private per CPU
Access	Uniform (UMA)	Non-uniform (NUMA)
Communication	Implicit (shared vars)	Explicit (MPI send/rcv)
Data distribution	Not needed	User-controlled
Work distribution	#pragma omp for	User-specified
Scalability	~8–64 CPUs	Millions of processes
Programming	OpenMP	MPI
Fault tolerance	Poor	Better



Do 6 processes: → nodes 1 & node 2

DMP in Action — Dot Product with MPI

Array $a[100]$ split across 2 nodes — explicit data send + MPI_Reduce

Node 1 (Master / Root)

```
// Has full a[100]
MPI_Send(a[51..100], 50, Node2)
```

```
// Compute local partial sum
S1 = 0
for (i=1 to 50):
    S1 += a[i] * a[i]
```

```
// Gather both partial sums
MPI_Reduce(S1, &S, MPI_SUM)
print(S) // Final result
```

Key role: Master coordinates. Sends data to Node 2. Computes lower half. Collects final result via MPI_Reduce.

Node 2 (Worker)

```
// Receives b = a[51..100]
MPI_Recv(b, 50, Node1)
```

```
// Compute local partial sum
S2 = 0
for (i=1 to 50):
    S2 += b[i] * b[i]
```

```
// Send S2 back to master
MPI_Reduce(S2, &S, MPI_SUM)
```

Key role: Receives its data shard explicitly via MPI. Computes its partial sum. Sends result back. No shared memory.

MPI
→

Reduce

MPI — Message Passing Interface

The standard library for distributed memory communication

Point-to-Point

```
// Process 0 sends to Process 1
MPI_Send(&data, count, MPI_INT,
        dest=1, tag=0, comm);

// Process 1 receives
MPI_Recv(&data, count, MPI_INT,
        src=0, tag=0, comm, &st);
```

Collective Operations

MPI_Bcast	Root broadcasts data to ALL processes
MPI_Scatter	Root distributes chunks of array to each process
MPI_Gather	All processes send their chunk back to root
MPI_Reduce	Combine values (sum/max/min) from all → root
MPI_Allreduce	Like Reduce but ALL processes get the result
MPI_Barrier	All processes wait until every process arrives

MPI_Init() / **MPI_Finalize()** required at start/end. **MPI_Comm_rank()** → my process ID. **MPI_Comm_size()** → total processes.

§3

Hybrid Architectures

SMP within node + DMP across nodes + GPU Accelerators

Hybrid Architectures

Best of both worlds — SMP inside each node, MPI between nodes



Three Layers of Parallelism

Layer 1: Node Interconnect (DMP)

Multiple compute nodes connected by InfiniBand or Ethernet. MPI handles inter-node communication. Can scale to millions of processes.

Layer 2: Within-Node (SMP)

Each node has 2+ CPU sockets, each with multiple cores. OpenMP or threading handles intra-node parallelism. Shared memory within the node.

Layer 3: GPU Accelerators

Each node has 1–8 GPUs connected via PCIe or NVLink. CUDA (NVIDIA), HIP (AMD/ROCm), or OpenCL for GPU programming.

MPI + X Programming Models

MPI + OpenMP

(CPU threads)

MPI across nodes, OpenMP within node. Most common HPC combination.

MPI + CUDA

(NVIDIA GPU)

MPI across nodes, CUDA for GPU acceleration. Used in most AI clusters.

MPI + HIP

(AMD GPU)

CUDA-like programming for AMD ROCm GPUs. Frontier supercomputer uses this.

MPI + OpenCL

(Any GPU)

Vendor-agnostic GPU programming. Cross-platform but lower performance.

MPI + NCCL

(NVIDIA Collective)

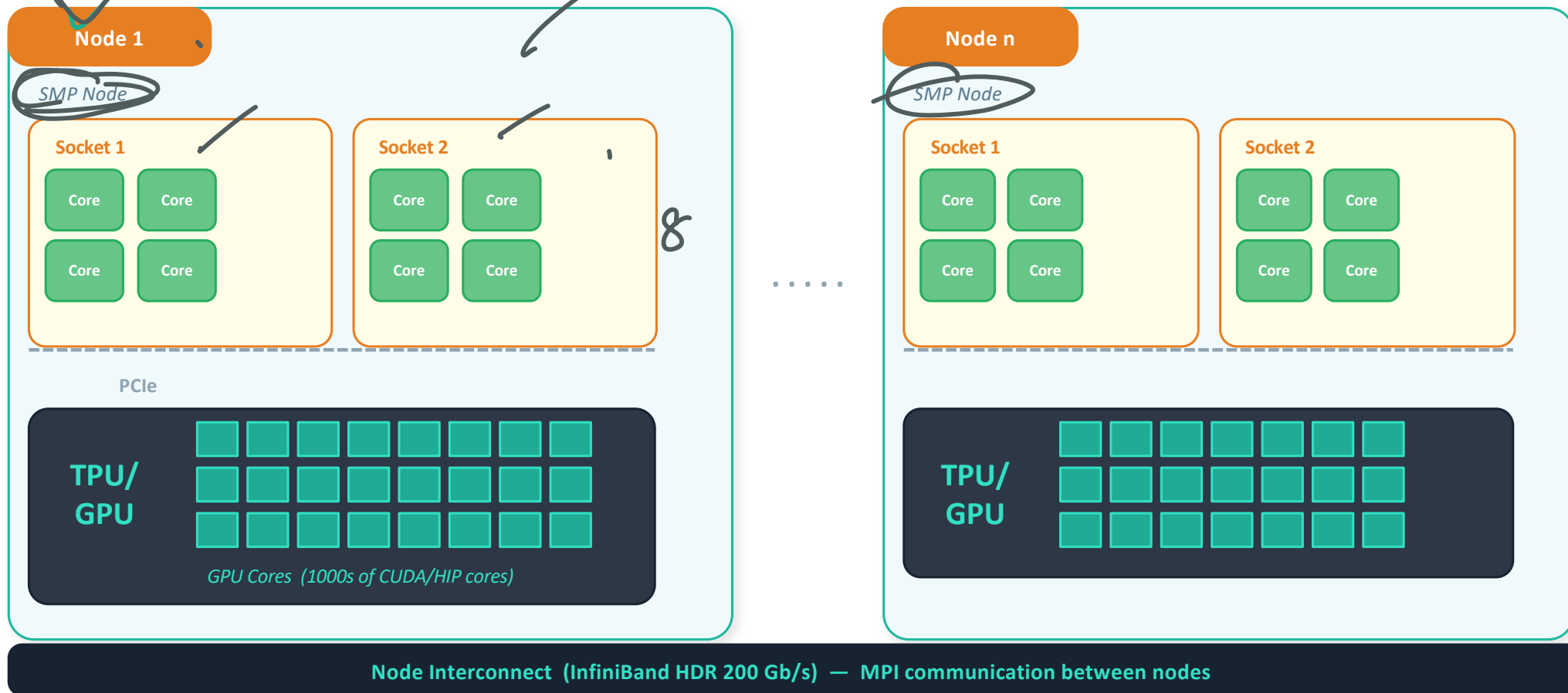
For deep learning — NCCL replaces MPI for GPU-to-GPU gradient sync.

Hybrid Node Architecture

AI Computer / cluster



SMP multi-core CPU node with GPU accelerator — connected via node interconnect



SMP vs. DMP vs. Hybrid — Full Comparison



Choose your parallel architecture based on your problem and hardware

Property	SMP	DMP	Hybrid (SMP+DMP+TPU/GPU)
Memory model	Shared — all CPUs see same RAM	Private — each CPU has own RAM	Both: shared within node, private between
Memory access	UMA — uniform	NUMA — non-uniform	Mixed: UMA intra-node, NUMA inter-node
Communication	Implicit (shared variables)	Explicit (MPI_Send/Recv)	MPI across nodes + shared mem within
Programming model	OpenMP directives	MPI library calls	MPI + OpenMP/CUDA/HIP
Scalability	~8–64 cores (bus bottleneck)	Millions of processes	Tens of thousands of nodes
Typical hardware	Workstation, laptop	Older clusters, cloud VMs	Modern supercomputers, TPU/GPU clusters
Data distribution	Not needed — global shared	User must explicitly distribute	User distributes data across nodes
Synchronisation	Implicit (barrier at region end)	Explicit (MPI_Barrier)	Both — openmp within, MPI across
Examples	Intel i9, AMD Ryzen (8 cores)	Param Pravega (IISc), Cori	Frontier (AMD GPU), Summit (NVIDIA GPU), GCP (TPU)
Best for	Fine-grained shared tasks	Coarse-grained distributed work	Large-scale ML/simulation

KEY TAKEAWAYS — Parallel Computer Architectures

SMP = shared memory + OpenMP All CPUs access one global memory. Use `#pragma omp parallel for` to split work across threads. No data distribution required.

DMP = distributed memory + MPI Each process owns private memory. Communicate explicitly via `MPI_Send/Recv/Reduce`. User controls both data AND work distribution.

Work decomp $\neq$ data decomp SMP decomposes work (loop iterations). DMP decomposes data (array chunks). Hybrid does both at different levels.

Hybrid = SMP $\times$ DMP $\times$ GPU Modern supercomputers layer all three: MPI across nodes, OpenMP within node, CUDA/HIP on GPU. Each level uses its own API.

Reduce operation is the glue Whether SMP (reduction clause) or DMP (`MPI_Reduce`), combining partial results is the critical synchronisation step.

Scalability ladder SMP $\rightarrow$ ~ 64 cores. DMP $\rightarrow$ $\sim 1\text{M}$ processes. Hybrid $\rightarrow$ Frontier (8.7 ExaFLOPS, $\sim 600\text{K}$ CPUs + 37K GPUs).